SPEC PHẦN MỀM — DỰ ÁN 1

Điều khiển nhiệt ẩm (Peltier) — Góc nhìn firmware

Tài liệu dành cho thành viên tập trung phần mềm (không yêu cầu kiến thức thiết kế phần cứng chi tiết)

April 8, 2026

Contents

1 Dự án 1 — Phần mềm điều khiển nhiệt ẩm (Peltier) 2

1.1 Mục tiêu phần mềm và giả định làm việc 2
1.1.1 Mục tiêu 2
1.1.2 Giả định (bạn không cần thiết kế mạch) 2
1.2 Nền tảng và công cụ phát triển 3
1.2.1 Dùng gì 3
1.2.2 Component / module trong firmware (gợi ý kiến trúc) . . . 3
1.3 Thiết kế phần mềm — luồng dữ liệu và trạng thái 4
1.3.1 Sơ đồ khối logic (phần mềm) 4
1.3.2 State machine / chu kỳ chính (gợi ý) 4
1.4 Thuật toán điều khiển (chi tiết cho lập trình) 4
1.4.1 Ngưỡng (có thể là tham số cấu hình) 4
1.4.2 Chống đóng/ngắt liên tục 5
1.4.3 Fail-safe phần mềm 5
1.5 Giao diện phần mềm với "thế giới bên ngoài" (abstract) 5
1.5.1 Đầu vào (input) 5
1.5.2 Đầu ra (output) — bạn cần deliver gì 5
1.6 Yêu cầu phi chức năng (phần mềm) 6
1.7 Tiêu chí nghiệm thu phía firmware (không cần đo bằng đồng hồ vạn năng) 6
1.8 Tích hợp với dự án 3 (mesh) — phạm vi phần mềm 6
1.9 Phụ lục: Bảng tham số mặc định (điền vào khi triển khai) 7

Chapter 1

Dự án 1 — Phần mềm điều khiển nhiệt ẩm (Peltier)

1.1 Mục tiêu phần mềm và giả định làm việc

1.1.1 Mục tiêu

Phần mềm chạy trên **ESP32** có nhiệm vụ:

- **Đọc định kỳ** nhiệt độ T và độ ẩm tương đối H từ cảm biến môi trường qua bus **I²C** (trong spec hệ thống: SHT35).
- **Tính toán** giá trị đã lọc (trung bình trượt) và áp dụng luật **On–Off có hysteresis** cùng **min ON/OFF time** để quyết định bật/tắt làm lạnh/hút ẩm.
- **Xuất tín hiệu điều khiển** qua một chân **GPIO** (mức cao/thấp) để bật/tắt khối công suất ngoài (bạn chỉ cần biết: "bật = Peltier được cấp điện", "tắt = ngắt"; chi tiết MOSFET/relay do nhóm phần cứng đảm nhiệm).
- **Ghi log / telemetry** (UART và/hoặc mesh) để debug và giám sát.
- **Fail-safe**: khi cảm biến lỗi hoặc mất lệnh từ mạng (nếu bật tích hợp mesh), chuyển về trạng thái an toàn (tắt đầu ra).

1.1.2 Giả định (bạn không cần thiết kế mạch)

- Địa chỉ I²C, chân SDA/SCL và chân GPIO điều khiển được **cố định trong code hoặc Kconfig** theo bảng wiring do nhóm phần cứng cung cấp.

- Driver cảm biến có thể dùng thư viện có sẵn (component trong repo) hoặc đọc thanh ghi theo datasheet — phần mềm chỉ cần API kiểu `read_temp_humidity(&T, &H)`.
- Phần cứng đảm bảo mức logic 3.3V và kết nối đúng; firmware không xử lý bảo vệ quá dòng trên bus nguồn 12V.

1.2 Nền tảng và công cụ phát triển

1.2.1 Dùng gì

- **SDK**: ESP-IDF (khuyến nghị phiên bản tương thích với branch dự án, ví dụ 5.x).
- **Ngôn ngữ**: C (chuẩn embedded ESP32).
- **Hệ điều hành nhẹ**: FreeRTOS (task, queue, timer software) — có thể dùng 1 task chính vòng lặp định kỳ hoặc tách task đọc sensor / task điều khiển.
- **Cấu hình build**: `menuconfig` / `sdkconfig` / `Kconfig` cho ngưỡng mặc định, chân GPIO, bật/tắt mesh.
- **Lưu cấu hình runtime** (tuỳ chọn): NVS để lưu ngưỡng T/H, min ON/OFF sau khi cấp nguồn lại.

1.2.2 Component / module trong firmware (gợi ý kiến trúc)

- `env_sensor`: khởi tạo I^2C, đọc SHT35, trả về T, H hoặc mã lỗi.
- `peltier_ctrl`: trạng thái bật/tắt, áp dụng hysteresis, min ON/OFF, lọc số.
- `actuator_gpio`: bọc `gpio_set_level(GPIO_PEL_EN, 0/1)` — một chỗ duy nhất để đổi chân nếu hardware đổi mapping.
- `app_main` hoặc task điều phối: lịch đọc sensor, gọi bộ điều khiển, log.
- (Tuỳ chọn) `mesh_client`: nhận lệnh setpoint, gửi telemetry (theo dự án 3).

1.3 Thiết kế phần mềm — luồng dữ liệu và trạng thái

1.3.1 Sơ đồ khối logic (phần mềm)

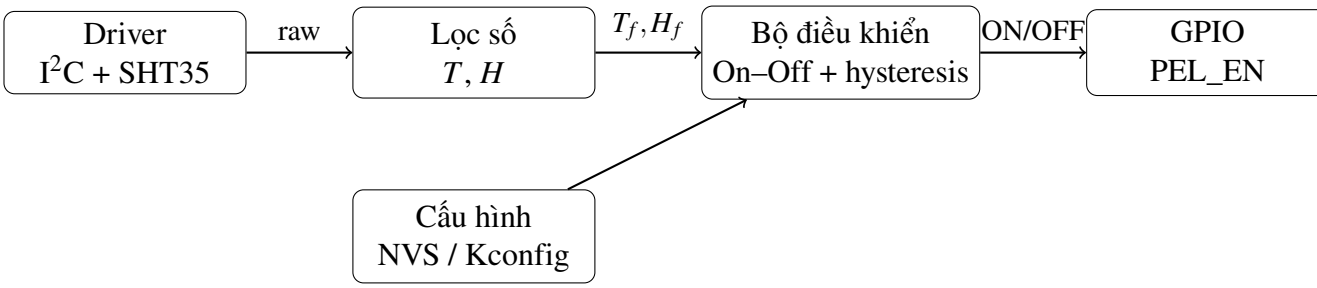


Figure 1.1: Luồng xử lý phần mềm (không mô tả chi tiết mạch công suất)

1.3.2 State machine / chu kỳ chính (gợi ý)

1. Khởi tạo: I^{2}C, GPIO đầu ra, (tuỳ chọn) NVS, UART log.
2. Vòng lặp định kỳ (ví dụ mỗi 500–1000 ms):
   (a) Đọc T, H từ sensor; nếu lỗi $\rightarrow$ tắt đầu ra, set cờ lỗi, log.
   (b) Cập nhật bộ đệm lọc (moving average 5–10 mẫu).
   (c) Nếu có cảm biến quá nhiệt tuỳ chọn $\rightarrow$ kiểm tra ngưỡng (nếu tích hợp trong firmware).
   (d) Áp dụng luật bật/tắt + min ON/OFF.
   (e) Ghi GPIO và log một dòng trạng thái.
   (f) (Tuỳ chọn) gửi gói telemetry lên mesh.

1.4 Thuật toán điều khiển (chi tiết cho lập trình)

1.4.1 Ngưỡng (có thể là tham số cấu hình)

- **Bật** làm lạnh/hút ẩm khi: $(T_f > 25°C)$ **hoặc** $(H_f > 65\%)$.
- **Tắt** khi: $(T_f < 20°C)$ **và** $(H_f < 60\%)$.

Ở đây T_f, H_f là giá trị sau lọc.

1.4.2 Chống đóng/ngắt liên tục

- **Lọc số**: trung bình trượt trên N mẫu ($N = 5 \dots 10$), cập nhật mỗi lần đọc.
- **Min ON time**: sau khi bật, không được tắt trước $t_{\text{on,min}}$ (ví dụ 10–30 s).
- **Min OFF time**: sau khi tắt, không được bật lại trước $t_{\text{off,min}}$ (ví dụ 10–30 s).

Cài đặt cụ thể nên là hằng số có tên rõ ràng hoặc biến NVS.

1.4.3 Fail-safe phần mềm

- **Lỗi đọc sensor** (timeout, CRC, giá trị NaN): coi như không tin cậy → **tắt** GPIO Peltier, log lỗi, có thể nháy LED debug nếu có.
- **Mất heartbeat / lệnh từ mesh** (nếu dùng): sau t_{timeout} (ví dụ 3–5 s) không nhận lệnh hợp lệ → tắt Peltier (theo spec hệ thống).

1.5 Giao diện phần mềm với "thế giới bên ngoài" (abstract)

1.5.1 Đầu vào (input)

- **I²C**: dữ liệu T, H từ driver SHT35.
- **Cấu hình**: file `sdkconfig` / Kconfig; có thể ghi đè bằng NVS sau lần chạy đầu.
- (Tuỳ chọn) **Mesh / UART**: nhận lệnh đổi ngưỡng, enable/disable từ remote.

1.5.2 Đầu ra (output) — bạn cần deliver gì

- **GPIO**: một bit logic điều khiển đường bật/tắt Peltier (tên symbolic ví dụ `GPIO_PEL_EN`).
- **UART log** (định dạng gợi ý, một dòng mỗi chu kỳ hoặc khi đổi trạng thái):

```
[timestamp_ms] T=24.80 H=62.1 PEL=ON filt_ok=1 err=0
```

- **Telemetry** (nếu tích hợp dự án 3): JSON hoặc struct cố định gồm ít nhất: `temp`, `hum`, `peltier_on`, `sensor_ok`, `err_code`.

1.6 Yêu cầu phi chức năng (phần mềm)

- **Chu kỳ đọc sensor**: 1–2 Hz (ổn định, không block lâu trên I^2C).
- **Độ trễ phản ứng**: sau khi điều kiện vượt ngưỡng, hành vi bật/tắt vẫn tuân min ON/OFF (có thể chấp nhận trễ thêm vài chu kỳ do lọc).
- **Không treo**: tránh busy-wait dài; dùng timer FreeRTOS hoặc `vTaskDelay` cho chu kỳ chính.
- (Khuyến nghị) **Watchdog task**: đăng ký task quan trọng với TWDT nếu dự án yêu cầu độ tin cậy cao.

1.7 Tiêu chí nghiệm thu phía firmware (không cần đo bằng đồng hồ vạn năng)

- Build project thành công, flash lên board ESP32 do nhóm cung cấp.
- Log UART hiển thị T, H thay đổi hợp lý khi môi trường thay đổi (hoặc mock sensor trong unit test nếu có).
- Khi T, H (đã lọc) vượt ngưỡng bật $\rightarrow$ sau min OFF (nếu đang tắt), GPIO chuyển sang trạng thái bật; khi đạt vùng tắt $\rightarrow$ sau min ON, GPIO tắt.
- Ngắt kết nối sensor (hoặc mô phỏng lỗi driver) $\rightarrow$ firmware tắt đầu ra và báo lỗi trong log.
- (Nếu có mesh) mất kết nối sau timeout $\rightarrow$ đầu ra về an toàn, log rõ ràng.

1.8 Tích hợp với dự án 3 (mesh) — phạm vi phần mềm

Firmware node Peltier có thể:

- Đăng ký làm node con, nhận bản tin chứa ngưỡng $T_{\min}, T_{\max}, H_{\min}, H_{\max}$ hoặc `enable`.
- Gửi định kỳ telemetry trùng khớp với dữ liệu đang log UART.

Chi tiết định dạng bản tin tham chiếu tài liệu dự án 3; phần mềm dự án 1 chỉ cần implement parser tối thiểu và callback cập nhật tham số điều khiển.

1.9 Phụ lục: Bảng tham số mặc định (điền vào khi triển khai)

Table 1.1: Tham số firmware — dự án 1 (mẫu)

Tham số	**Gợi ý**	**Ghi chú**
`READ_PERIOD_MS`	500–1000	Chu kỳ vòng lặp chính
`FILTER_LEN`	5–10	Số mẫu trung bình trượt
`T_ON`, `H_ON`	25°C, 65%	Ngưỡng bật (OR)
`T_OFF`, `H_OFF`	20°C, 60%	Ngưỡng tắt (AND)
`MIN_ON_S`, `MIN_OFF_S`	10–30	Giới hạn thời gian bật/tắt
`GPIO_PEL_EN`	Theo wiring	Chỉ đổi một chỗ trong code